

Model-Generic Repository

- **10-1:Generic Repository(資料存取層)**
 - 為何使用Generic Repository.....10-2
 - 使用Generic Repository.....10-4
- **10-2: .Net Core的Repository**
 - .Net Core使用Repository.....10-8
 - .Net Core使用Generic Repository.....10-12

Module 10

為何使用 Generic Repository -1

- **MVC 架構強調職責分離**：Controller 負責 接收請求與流程協調，不應塞滿資料存取細節。為了落實分工與提升維護性，通常會將 資料存取（Data Access）的程式碼抽離，集中管理。
- **Generic Repository 是一種常見的資料存取抽象化做法**：以 Entity Framework Core（DbContext）為基礎，透過泛型（Generic）封裝共用 CRUD 操作，使不同 Entity 可重用同一套存取邏輯。
- **Generic Repository 把「資料庫操作」從 Controller 抽離**，讓程式更乾淨、更好維護。

```
public ActionResult Index()
{
    return View(new Repository<Products>().GetAll());
}
```

為何使用 Generic Repository -2

- 優點：
 - **Controller** 更精簡：只負責呼叫服務/**Repository**，不直接操作 **DbContext**
 - 降低重複程式碼：共用 **CRUD** 寫一次即可套用多個 **Entity**
 - 提升可測試性：可用介面（**Interface**）替換實作，方便單元測試
 - 集中維護：資料存取邏輯一致，變更更容易控管
- **Repository**單純只做**CRUD**查詢資料，不負責商業邏輯、不做流程判斷。

使用 Generic Repository -1

- 製作 IRepository 的 Interface 介面。

```
interface IRepository<Table>
{
    IEnumerable<Table> GetAll();
    void Create(Table _entity);
    void Update(Table _entity);
    Table GetByID(int ID);
    void Delete(int ID);
}
```

- 使用 Interface 介面的目的，是日後若要建立其他資料庫或連接方式的 Repository Model 時，更方便建立欄位方法且具一致性。

使用 Generic Repository -2

- 製作Repository Model，實作剛剛製作的IRepository介面，利用 where : class 表示 < > 內的泛型參數為 class。

```
public class StoreRepository<Table> : IRepository<Table> where Table : class
```

- 實作介面，並完成邏輯與連接程式。

```
public class StoreRepository<Table> : IRepository<Table> where Table : class
{
    public void Create(Table _entity)
    {
        throw new NotImplementedException();
    }

    public void Delete(Table _entity)
    {
        throw new NotImplementedException();
    }
}
```

使用 Generic Repository -3

```
public class StoreRepository<Table> : IRepository<Table> where Table : class
```

```
{  
    StoreDBEntities db = null;    // 底層為DbContext
```

```
    DbSet<Table> dbTable = null; // EntityFramework的屬性類別
```

```
    public StoreRepository()
```

```
    {  
        db = new StoreDBEntities(); // 建立StoreDBEntities的實體  
        dbTable = db.Set<Table>();  
    }
```

```
    public void Create(Table _entity)
```

```
    {  
        dbTable.Add(_entity); // 新增資料  
        db.SaveChanges();  
    }
```

```
    public Table GetByID(int ID)
```

```
    {  
        return dbTable.Find(ID); // 讀取單筆資料  
    }
```

```
    public void Delete(int ID)
```

```
    {  
        dbTable.Remove(GetByID(ID)); // 刪除單筆資料  
        db.SaveChanges();  
    }
```

```
    public IEnumerable<Table> GetAll()
```

```
    {  
        return dbTable; // 讀取全部資料  
    }
```

```
    public void Update(Table _entity)
```

```
    {  
        db.Entry<Table>(_entity).State = EntityState.Modified;  
        db.SaveChanges();  
    }
```

```
}
```

使用Generic Repository -4

- 在Controller使用Generic Repository。

```
public class CitiesController : Controller
{
    StoreRepository<City> dbCity = new StoreRepository<City>();
    public ActionResult Index()
    {
        return View(dbCity.GetAll());
    }
    public ActionResult Details(int? id)
    {
        City city = dbCity.GetByID((int)id);
        return View(city);
    }
}
```

.Net Core使用Repository -1

- 建立Repository Interface :

```
public interface INorthwindShipperRepository
{
    2 個參考
    IEnumerable<Shipper> GetAll();
    1 個參考
    Shipper? GetById(int id);
    1 個參考
    void Add(Shipper shipper);
    1 個參考
    void SaveChanges();
    1 個參考
    bool Delete(int id);
}
```


.Net Core使用Repository -2

- 建立Repository類別：

```
public class NorthwindShipperRepository : INorthwindShipperRepository
{
    private readonly NorthwindDbContext.NorthwindDbContext _dbContext;
    0 個參考
    public NorthwindShipperRepository(NorthwindDbContext.NorthwindDbContext dbContext)
    {
        _dbContext = dbContext;
    }
    1 個參考
    public void SaveChanges()
    {
        _dbContext.SaveChanges();
    }
    1 個參考
    public void Add(Shipper shipper)
    {
        _dbContext.Shippers.Add(shipper);
    }
    1 個參考
    public bool Delete(int id)
    {
        var qShipper = _dbContext.Shippers.FirstOrDefault(s => s.ShipperId == id);
        if (qShipper != null)
        {
            _dbContext.Shippers.Remove(qShipper);
        }
        return true;
    }
}
```

.Net Core使用Repository -3

- 建立DI繫結(Program.cs) :

```
// Add services to the container.  
builder.Services.AddControllersWithViews();  
  
builder.Services.AddDbContext<NorthwindDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("NorthwindConnectionStrings")));  
  
builder.Services.AddScoped<INorthwindShipperRepository, NorthwindShipperRepository>();  
  
builder.Services.AddScoped<INorthwindShipperService, NorthwindShipperService>();  
  
var app = builder.Build();
```

.Net Core使用Repository -4

- 使用Repository :

```
public class ShippersController : Controller
{
    private readonly INorthwindShipperRepository _shipperRepository;

    0 個參考
    public ShippersController(INorthwindShipperRepository shipperRepository)
    {
        _shipperRepository = shipperRepository;
    }

    0 個參考
    public IActionResult Index()
    {
        return View(_shipperRepository.GetAll());
    }
}
```

.Net Core使用Generic Repository -1

- 建立可供所有Table資料表使用的Interface(使用泛型)：

```
public interface IGenericRepository<Table> where Table : class
{
    1 個參考
    Table GetById(object id);
    2 個參考
    IEnumerable<Table> GetAll();

    1 個參考
    void Add(Table entity);
    1 個參考
    void Update(Table entity);
    1 個參考
    void Remove(Table entity);

    1 個參考
    void SaveChanges();
}
```

.Net Core使用Generic Repository -2

- 建立指定資料庫使用的Repository :

```
public class NorthwindGenericRepository<Table> : IGenericRepository<Table> where Table : class
{
    public readonly NorthwindDbContext.NorthwindDbContext _dbContext;
    public readonly DbSet<Table> _dbSet;
    1 個參考
    public NorthwindGenericRepository(NorthwindDbContext.NorthwindDbContext dbContext)
    {
        _dbContext = dbContext;
        _dbSet = _dbContext.Set<Table>();
    }

    2 個參考
    public IEnumerable<Table> GetAll()
    {
        return _dbSet.AsNoTracking();
    }
    1 個參考
    public void Add(Table entity)
    {
        _dbSet.Add(entity);
    }
}
```

.Net Core使用Generic Repository -3

- 建立DI繫結並指定將處理的資料表模型(Program.cs) :

```
// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<NorthwindDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("NorthwindConnectionStrings")));

builder.Services.AddScoped<INorthwindShipperRepository, NorthwindShipperRepository>();

builder.Services.AddScoped(typeof(IGenericRepository<>), typeof(NorthwindGenericRepository<>));
builder.Services.AddScoped<IGenericRepository<Shipper>, NorthwindGenericRepository<Shipper>>();

builder.Services.AddScoped<INorthwindShipperService, NorthwindShipperService>();

var app = builder.Build();
```

.Net Core使用Generic Repository -4

- 使用Repository :

```
public class ShippersController : Controller
{
    private readonly IGenericRepository<Shipper> _shipperRepository;
    0 個參考
    public ShippersController(IGenericRepository<Shipper> shipperRepository)
    {
        _shipperRepository = shipperRepository;
    }
    0 個參考
    public IActionResult Index()
    {
        return View(_shipperRepository.GetAll());
    }
}
```